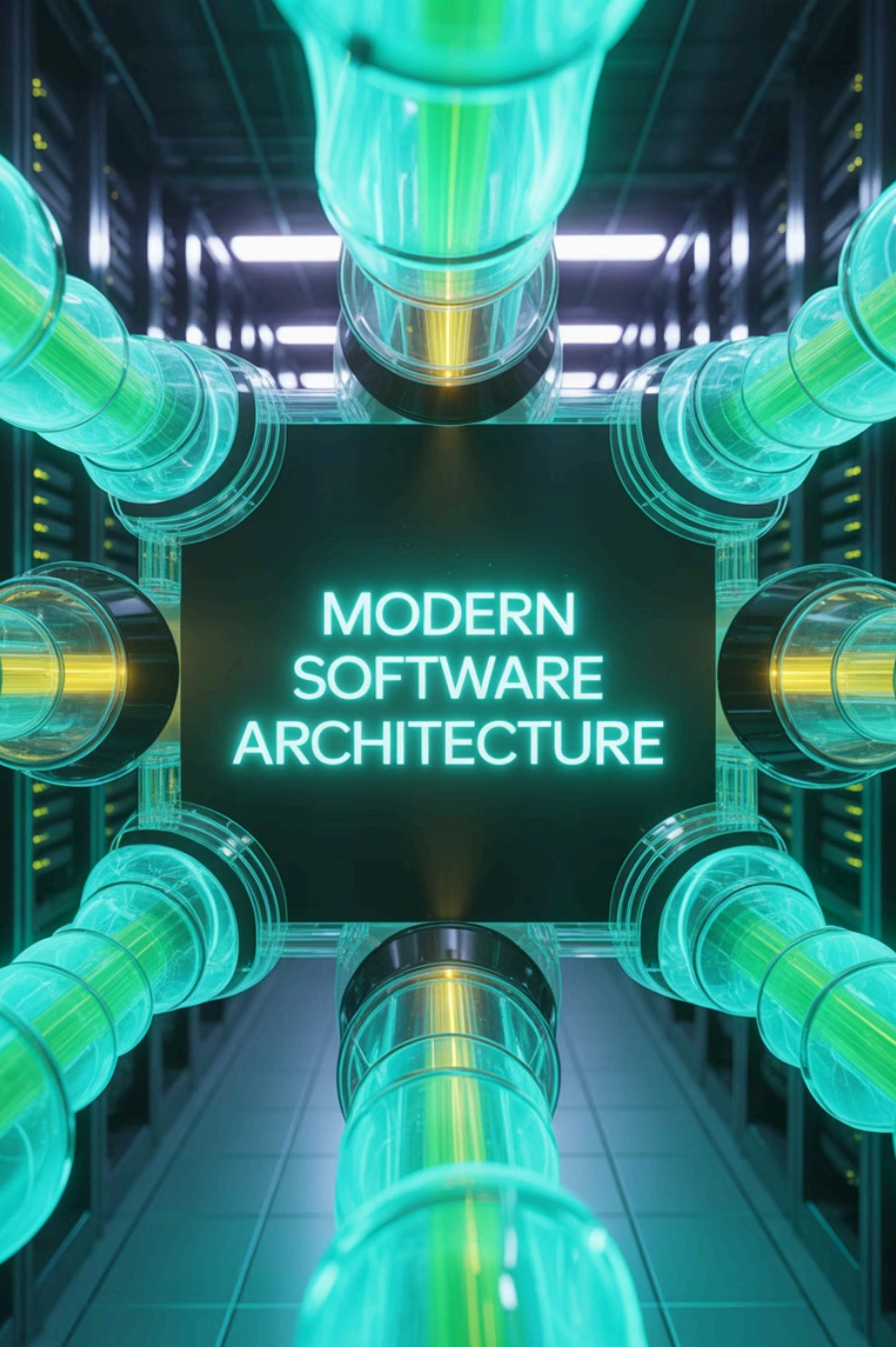


Deep Dive into RxJS and Signals

Architecture, Flows, and Critical Gotchas for Senior Developers

Empowering senior developers to master advanced state architecture and reactive flow patterns. This deep dive explores how RxJS and Angular Signals work synergistically to create robust, maintainable applications.

Key Message: RxJS and Angular Signals are Better Together



MODERN
SOFTWARE
ARCHITECTURE

Reactive Programming

Teulice orotupry 20/12

What is a Signal?

A value + change notification system that provides fine-grained control over Change Detection. Angular's strategic step toward eliminating Zone.js dependency.

Signal Reading Pattern

Always call the getter function: `quantity()`. Common mistake to avoid: `quantity` displays the Signal object, not the value.

Signals represent Angular's new reactivity model, offering developers unprecedented control over when and how components respond to state changes. This foundation enables more predictable performance patterns and simplified debugging workflows.



Writable and Computed Signals Architecture

Writable Signals

The state storage and mutation layer using `signal()`:

- `.set(newValue)`: Complete value replacement
- `.update(oldValue => newValue)`: Updates based on current value
- `.mutate(obj => obj.property = newValue)`: In-place object/array modification

Computed Signals

Reactive, read-only derived values using `computed()`:

- Automatic memoization - recalculates only when dependencies change
- Perfect for read-only selectors and calculations
- Maintains referential stability for performance

❏ **Critical Rule for Effects:** Use `effect()` only for side effects (logging, external API calls). Never use effects to change state - risk of infinite loops!

Redux-Like State Architecture Pattern

01

Private Reactive Source

Service maintains: `private state = signal(initialState)` as the single source of truth

02

Public Selectors

Expose computed read-only access: `public readonly isLoading = computed(() => this.state().isLoading)`

03

Async Decision Point

Choose RxJS for HTTP requests and async flows. Use Signals for synchronous filtering/transformation of existing data.



Action Handling and State Reducers

Async Action Processing

Utilize RxJS Subject as action entry point:

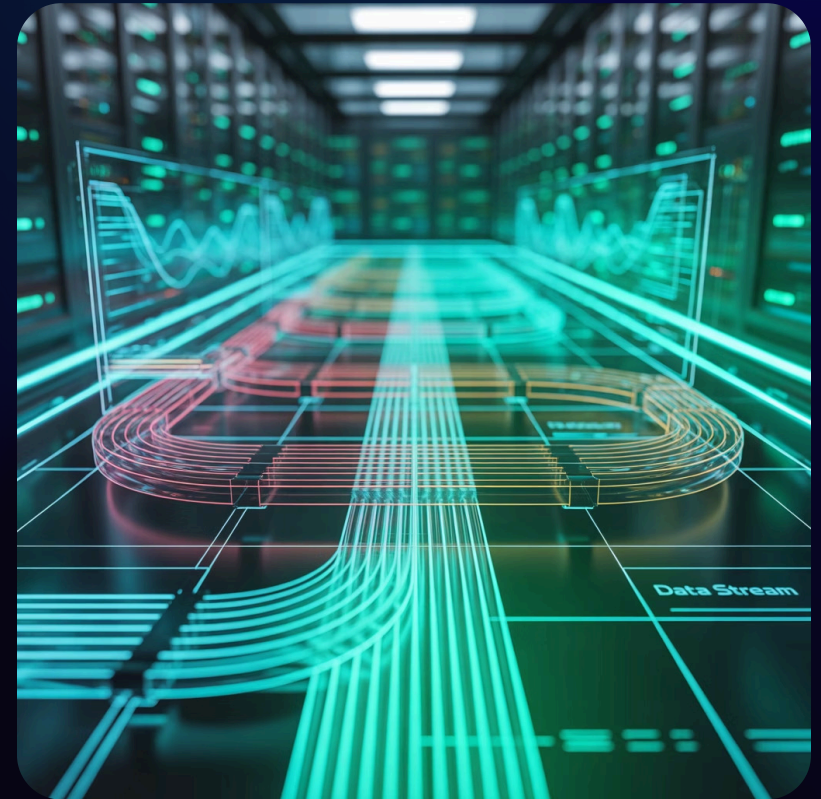
- Component calls `service.selectItem(id)`
- Service executes `subject.next(id)`
- Subject triggers RxJS pipeline (HTTP, error handling, etc.)

State Updaters (Reducers)

Private service methods update state:

```
this.state.update(oldState => ({ ...oldState, items: newItems })))
```

Critical: Always create new instances (spread operator) for objects/arrays so signals detect changes.



Retrieve Related Data: The Wicked Pipeline

Problem: Chained data retrieval (Username → UserID → UserPosts) creates complex async dependencies.

Solution: RxJS Subject-driven chain of switchMap operators:

```
action$.pipe(  
  switchMap(val => getFirstData(val)),  
  switchMap(data1 => getSecondData(data1))  
)
```

Senior Refactoring Tip

Break long pipelines into smaller, reusable methods (getUserID(), getPostsForUser()). This enables centralized error handling and improves testability.



Higher-Order Mapping: Critical Architectural Decision

Choosing the right operator for flattening Observables within Observables impacts application behavior significantly.

switchMap (Cancellation)

Use: User input (search, selection)

Behavior: Cancels previous inner observable when new one arrives

Result: Only the latest action matters

concatMap (Ordering)

Use: Sequential execution, waits for previous completion

Behavior: Operations execute one by one

Risk: Slow operations block; result order may be "stale"

mergeMap (Concurrency)

Use: Parallel execution of multiple operations

Behavior: All operations start simultaneously

Risk: Results arrive in unordered fashion

Retrieve Many Related Data Patterns

Problem: One-to-Many Relationships

Source Observable emits object containing array of IDs/URLs (e.g., `vehicle.films: string[ ]`). Need HTTP GET for each.

Solution 1: `forkJoin`

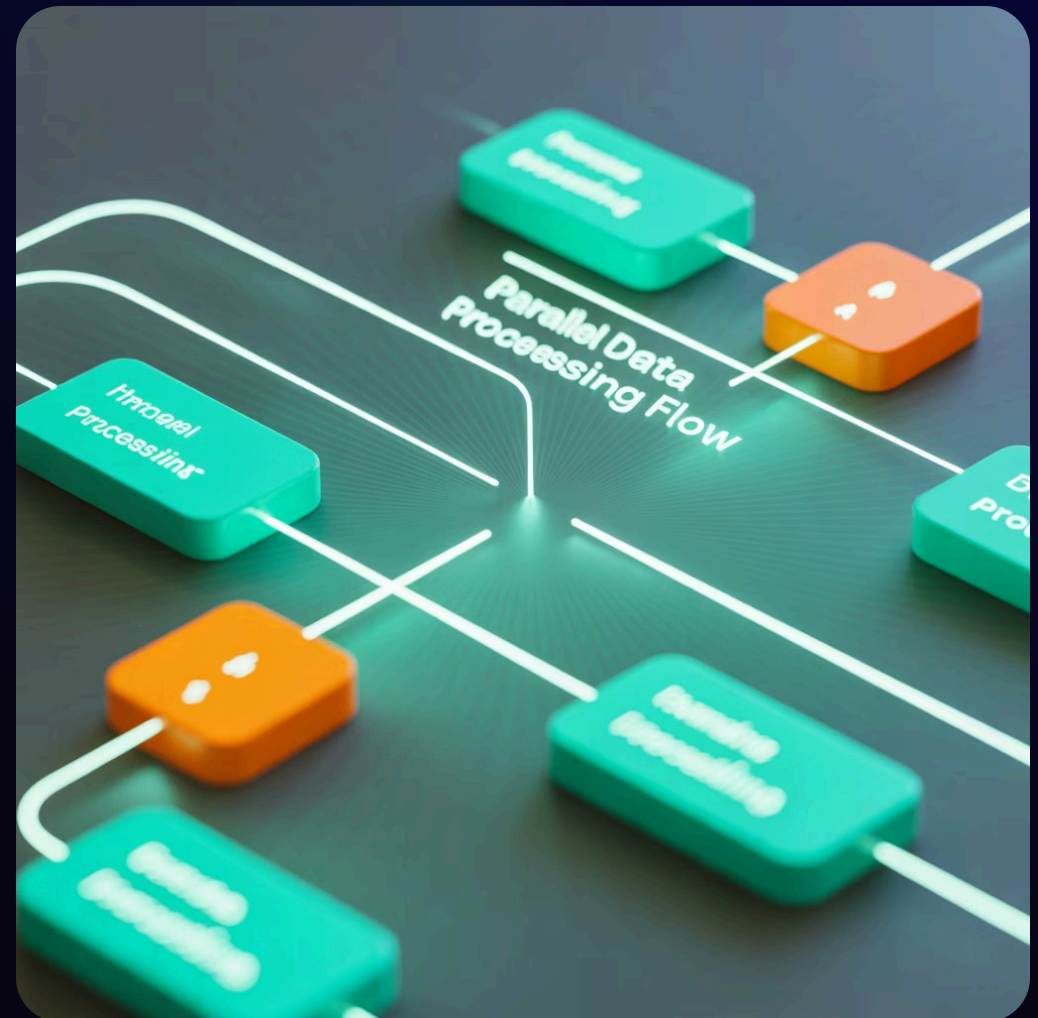
- Accepts array of inner observables (HTTP calls)
- Waits for all to complete
- Emits single array with last value from each observable
- Perfect for collecting all results from parallel, one-time operations

Solution 2: Alternative Pattern

`from(urlArray)`: Converts URL array to observable

`mergeMap(url => http.get(url))`: Parallel HTTP execution for each URL

`toArray()`: Collects all individual emissions into single array before stream completion



Golden Rule: subscribe → unsubscribe

Rule #1: If you subscribe, you must unsubscribe to prevent memory leaks and unexpected behavior.



takeUntilDestroyed() (TUD) - Angular v16+

Automatically completes observable when component/service is destroyed. Essential for managing subscription lifecycles in modern Angular applications.



TUD Appropriate Usage

Long-running streams (Interval, Subject) and HTTP GET requests where cancellation on navigation is desired for performance optimization.



TUD Critical Gotcha: HTTP Mutations



WARNING

NEVER use TUD with HTTP PUT, POST, DELETE operations!

The Problem

TUD completes the observable before the response returns. The subscribe handler (state/UI update) **never executes**. Users expect mutation to complete, but get silent data loss in UI.

The Solution

Let mutation observables complete naturally. Use TUD only for read operations and long-running streams where cancellation is beneficial.



take(1) Operator and Pipeline Ordering

take(1) Redundancy

No need to use `take(1)` on standard HTTP requests. Angular HTTP streams are one-and-done: they emit value once and complete automatically.

TUD and Operation Order

Critical: TUD only affects operators above it in the chain.

Risk: If TUD precedes `switchMap` that creates inner observable (e.g., `interval`), the inner observable may continue running (Memory Leak).

Rule: TUD must be **after operators you want to shut down** (`switchMap`), but **before operators waiting for stream completion** (`toArray`, `count`).



Stream Sharing: share vs shareReplay

Multicasting enables multiple subscribers to share the same source (e.g., HTTP call), preventing re-execution of source logic.

1

share() Gotcha

With one-and-done observables (HTTP), refCount immediately drops to zero. Late subscribers trigger new source execution (duplicate HTTP call).

2

shareReplay() Solution

Caches last value(s) and replays to late subscribers. Prevents re-execution of source logic, ensuring efficient resource usage.

3

shareReplay Memory Leak

Default: shareReplay() has refCount: false. With non-completing streams (interval), source continues forever even when all unsubscribe.

4

Senior Fix

`shareReplay({ refCount: true })` ensures source stream terminates when last subscriber unsubscribes.

Shape on Action Pattern: Filtering and Combining

Problem: User action shapes/filters existing data without new HTTP request.

RxJS Solution: combineLatest + map

```
combineLatest([AllProducts$,  
ProductSelectedAction$])  
  .pipe(  
    map(([products,  
selectedId]) =>  
      products.find(p => p.id  
=== selectedId)  
    )  
  )
```

When to use: When data exists in Observables

Lookup Reference Property: Object has categoryId, but UI needs categoryName. Use combineLatest to merge data sources and map relationships.

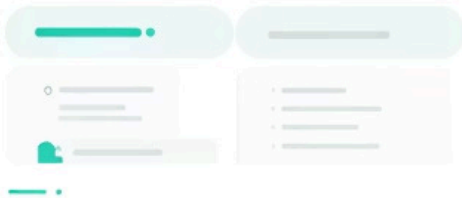
Signals Solution: Computed Signal

```
selectedProduct =  
computed(() =>  
  allProducts().find(p =>  
    p.id() === selectedId()  
  )  
)
```

When to use: When data exists in Signals; simpler and more efficient for synchronous state



Search



Search search routes



Project files



Meeting notes



Meeting notes



Contact details



Contact details



Autocomplete Pipeline: Multiple Action Synthesis

Problem: Single output stream (autocomplete list) reacts to multiple different async actions (text input, click, focus, clear).

01

Multiple Action Subjects

Create Subject for each action: Focus\$, Click\$, Clear\$, TextInput\$

02

Action Merging (OR Logic)

`merge(Focus$, Click$, Clear$, DebouncedText$)` - merge emits value as soon as any source emits

03

Input Optimization

On TextInput\$ stream: `debounceTime(200) → distinctUntilChanged()`

04

Final Filtering

`combineLatest([operationStream, masterCategories$])` with filtering logic

Architectural Decision Matrix



RxJS Domain

Asynchronous operations, complex flows, HTTP requests, external events, time-based operations



Signals Domain

Local state, derived values (Computed Signals), reactions to internal state changes. Maximize memoization benefits.

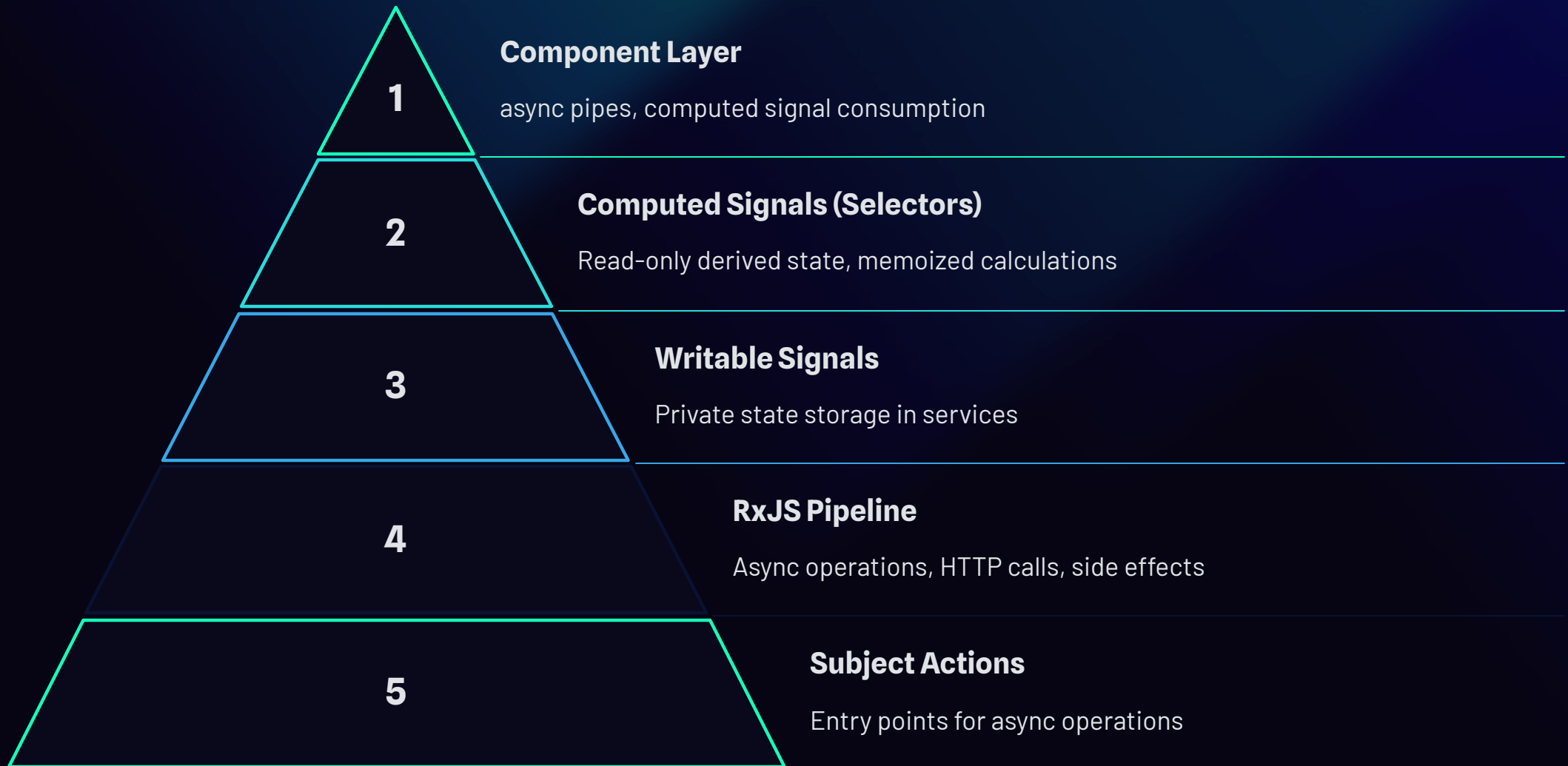


async Pipe Usage

Use async pipe in templates. Automatic subscribe/unsubscribe eliminates memory leaks and simplifies code management.

The key to successful architecture lies in understanding when to leverage each tool's strengths rather than forcing one pattern everywhere.

State Management Hierarchy





Memory Management Best Practices

1

Subscription Creation

Always consider lifecycle when creating subscriptions. Use reactive patterns to minimize manual subscription management.

2

Automatic Cleanup

Leverage `takeUntilDestroyed()`, `async pipe`, and computed signals for automatic resource management.

3

Manual Cleanup

For complex cases, implement `OnDestroy` with explicit `unsubscribe()` calls using subscription management patterns.

4

Testing & Monitoring

Use browser dev tools to monitor memory usage and identify subscription leaks during development.

Error Handling Strategies

RxJS Error Patterns

- `catchError` for graceful degradation
- `retry` and `retryWhen` for transient failures
- `throwError` for explicit error propagation
- Error handling at appropriate pipeline levels

Signal Error Patterns

- Error state in signal: `{ data, error, loading }`
- Computed signals can handle error states
- Effects can trigger error notifications



Robust applications require comprehensive error handling at every level of the reactive stack. Plan error scenarios during architecture design, not as an afterthought.

Performance Optimization Techniques



Signal Memoization

Computed signals automatically memoize results. Structure computations to maximize cache hits and minimize recalculation overhead.



Stream Sharing

Use `shareReplay()` strategically to prevent duplicate HTTP requests. Balance memory usage with computation savings.



Operator Optimization

Choose operators wisely. `debounceTime()` for user input, `distinctUntilChanged()` to prevent unnecessary emissions, `startWith()` for initial values.



State Structure

Normalize state structure. Keep related data co-located in signals while avoiding deeply nested structures that trigger excessive change detection.

Q&A and Final Thoughts

Key Takeaways

- RxJS and Signals complement each other perfectly
- Choose the right tool for each architectural layer
- Memory management is critical for production apps
- Error handling should be designed, not added later

Questions for Discussion

- How do you handle complex state synchronization?
- What patterns work best in your current projects?
- Where have you encountered the most challenging gotchas?
- How do you balance performance with maintainability?

The journey to mastering reactive patterns is ongoing. These patterns and practices will evolve as Angular continues to mature its reactive model. Stay curious, keep experimenting, and share your discoveries with the community.

